SEGUNDO DEPARTAMENTAL- EJERCICIO 10- LISTA DINÁMICA

En esta práctica desarrollamos un programa que permite implementar y administrar una lista dinámica utilizando la Biblioteca Estándar de Plantillas (STL). A diferencia de nuestra implementación anterior, en la que creamos manualmente los nodos y utilizamos apuntadores para conectarlos, en esta ocasión utilizamos el contenedor list<int>, que nos proporciona C++ para administrar automáticamente una lista de elementos enteros. Nuestro programa permite insertar números al inicio y al final, eliminar la primera aparición de un valor, buscar elementos, mostrar el contenido de la lista, verificar si está vacía y consultar la cantidad de elementos almacenados. Además, organizamos las operaciones mediante una clase llamada Lista y un menú interactivo que facilita su utilización.

Código Completo:

```
#include <iostream>
#include <list>
using namespace std;

class Lista {
private:
    list<int> datos;

public:
    // Insertar al inicio
    void insertarInicio(int valor) {
        datos.push_front(valor);
        cout << "Elemento insertado al inicio.\n";
    }

    // Insertar al final
    void insertarFinal(int valor) {
        datos.push_back(valor);
        cout << "Elemento insertado al final.\n";
    }

    // Eliminar la primera aparicion de un valor
    void eliminar(int valor) {
        for (list<int>::iterator it = datos.begin();
             it != datos.end(); ++it) {

            if (*it == valor) {
                datos.erase(it);
                cout << "Elemento eliminado.\n";
                return;
            }
        }

        cout << "Elemento no encontrado.\n";
    }

    // Buscar un elemento
    void buscar(int valor) {
        int posicion = 1;

        for (list<int>::iterator it = datos.begin();
             it != datos.end(); ++it) {
```

```
            if (*it == valor) {
                cout << "Elemento encontrado en la posicion "
                     << posicion << ".\n";
                return;
            }

            posicion++;
        }

        cout << "Elemento no encontrado.\n";
    }

    // Mostrar todos los elementos
    void mostrar() {
        if (datos.empty()) {
            cout << "La lista esta vacia.\n";
            return;
        }

        cout << "\nElementos de la lista:\n";

        for (list<int>::iterator it = datos.begin();
             it != datos.end(); ++it) {
            cout << *it << " -> ";
        }

        cout << "FIN\n";
    }

    // Consultar si la lista esta vacia
    void estaVacia() {
        if (datos.empty())
            cout << "La lista esta vacia.\n";
        else
            cout << "La lista contiene elementos.\n";
    }

    // Mostrar la cantidad de elementos
    void cantidad() {
        cout << "Cantidad de elementos: "
             << datos.size() << endl;
    }
};

int main() {
    Lista lista;
    int opcion, valor;

    do {
        cout << "\n=== LISTA DINAMICA CON STL ===\n";
        cout << "1. Insertar al inicio\n";
        cout << "2. Insertar al final\n";
        cout << "3. Eliminar elemento\n";
        cout << "4. Buscar elemento\n";
        cout << "5. Mostrar lista\n";
        cout << "6. Verificar si esta vacia\n";
        cout << "7. Consultar cantidad de elementos\n";
        cout << "8. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
```

```
                cout << "Ingresa un numero: ";
                cin >> valor;
                lista.insertarInicio(valor);
                break;

            case 2:
                cout << "Ingresa un numero: ";
                cin >> valor;
                lista.insertarFinal(valor);
                break;

            case 3:
                cout << "Numero que deseas eliminar: ";
                cin >> valor;
                lista.eliminar(valor);
                break;

            case 4:
                cout << "Numero que deseas buscar: ";
                cin >> valor;
                lista.buscar(valor);
                break;

            case 5:
                lista.mostrar();
                break;

            case 6:
                lista.estaVacia();
                break;

            case 7:
                lista.cantidad();
                break;

            case 8:
                cout << "Programa finalizado.\n";
                break;

            default:
                cout << "Opcion no valida.\n";
        }

    } while (opcion != 8);

    return 0;
}
```

Explicación del Código:

1. Declaración de la clase Lista (class Lista)

Creamos la clase Lista, que se encarga de administrar todos los elementos de nuestra estructura, dentro de ella definimos los atributos y métodos necesarios para insertar, eliminar, buscar, mostrar y consultar información de la lista.

2. Declaración del contenedor (list<int> datos)

Con esta instrucción creamos una lista llamada datos, que almacena elementos de tipo entero (int), el contenedor list pertenece a la biblioteca STL y utiliza una estructura de

nodos doblemente enlazados que administra automáticamente la memoria y las conexiones entre sus elementos y al declararlo como privado, evitamos que pueda modificarse directamente desde fuera de nuestra clase.

3. Método para insertar al inicio (insertarInicio(int valor))

Creamos el método insertarInicio(), cuya función es agregar un nuevo número al principio de nuestra lista. Recibimos el número mediante el parámetro valor, la función push_front() inserta el elemento al inicio de la lista, desplazando conceptualmente los elementos anteriores hacia las siguientes posiciones.

4. Método para insertar al final (insertarFinal(int valor))

Creamos el método insertarFinal(), que permite agregar un nuevo elemento al final de nuestra lista, la función push_back() agrega el número recibido después del último elemento almacenado y finalmente, mostramos un mensaje indicando que el elemento fue insertado correctamente.

5. Método para eliminar elementos (eliminar(int valor))

Creamos el método eliminar(), cuya función es localizar y eliminar la primera aparición de un número dentro de nuestra lista, primero recibimos el número que queremos eliminar mediante el parámetro valor, para realizar esta operación utilizamos un ciclo for con un iterador que nos permite recorrer los elementos.

6. Declaración del iterador (list<int>::iterator it)

Dentro del ciclo declaramos un iterador de tipo list<int>::iterator:

list<int>::iterator it = datos.begin();

El iterador it funciona como una referencia que nos permite acceder y desplazarnos entre los elementos de nuestra lista y utilizamos datos.begin() para obtener la posición del primer elemento.

7. Condición del ciclo (it != datos.end())

Esto significa que el recorrido continúa mientras el iterador no haya llegado a la posición que representa el final de la lista. Es importante mencionar que datos.end() no apunta al último elemento, sino a la posición posterior al último.

8. Comparación del elemento (if (*it == valor))

Comparamos ese valor con el número recibido mediante valor, si ambos coinciden, significa que encontramos el elemento que queremos eliminar.

9. Eliminación del elemento (datos.erase(it))

Cuando encontramos el número solicitado, utilizamos, la función erase() elimina el elemento ubicado en la posición indicada por el iterador, después mostramos un mensaje de confirmación y utilizamos return para terminar el método. Esto permite que solamente eliminemos la primera aparición del número, incluso si existen otros elementos con el mismo valor.

10. Método para buscar elementos (buscar(int valor))

Creamos el método buscar(), cuya función es localizar un número dentro de nuestra lista y mostrar la posición en la que se encuentra, primero recibimos el número que queremos buscar mediante el parámetro valor.

11. Recorrido de la lista mediante un iterador (for)

Utilizamos nuevamente un ciclo for para recorrer todos los elementos:

```
for (list<int>::iterator it = datos.begin();
    it != datos.end(); ++it)
```

El iterador comienza en el primer elemento y avanza hasta llegar al final de la lista.

12. Comparación del elemento buscado (if (*it == valor))

Dentro del ciclo comparamos el contenido de la posición actual con el número que queremos encontrar, si ambos valores coinciden, significa que localizamos el elemento.

13. Actualización de la posición (posicion++)

Si el elemento actual no coincide con el número buscado, incrementamos la variable:

```
posicion++;
```

Esto permite que el siguiente elemento tenga una posición mayor en una unidad y el procedimiento se repite hasta encontrar el valor o terminar el recorrido.

14. Método para mostrar los elementos (mostrar())

Creamos el método mostrar(), cuya función es visualizar todos los números almacenados en nuestra lista, primero verificamos si el contenedor está vacío mediante: if (datos.empty())

La función empty() devuelve true cuando no existen elementos y false cuando la lista contiene uno o más valores.

15. Método para verificar si la lista está vacía (estaVacia())

Creamos el método estaVacia(), cuya función es comprobar si nuestra lista contiene elementos, utilizamos la función empty() dentro de una estructura if-else:

```
if (datos.empty())
    cout << "La lista esta vacia.\n";
else
    cout << "La lista contiene elementos.\n";
```

Si la lista no contiene elementos, mostramos el primer mensaje. De lo contrario, indicamos que existen elementos almacenados.

A diferencia de otros métodos, esta función no devuelve un valor bool, sino que muestra directamente el resultado en pantalla.

16. Método para consultar la cantidad de elementos (cantidad())

Implementamos el método cantidad(), que permite conocer cuántos elementos contiene nuestra lista, utilizamos la función: datos.size()

Esta función devuelve la cantidad de elementos almacenados en el contenedor.

Después mostramos el resultado mediante cout, de esta manera, el usuario puede conocer el tamaño actual de nuestra lista.

17. Función principal (int main())

Declaramos la función principal del programa, desde donde iniciamos su ejecución., dentro de ella creamos un objeto llamado lista, perteneciente a nuestra clase Lista.

También declaramos dos variables enteras:

- opcion: almacena la opción seleccionada por el usuario.
- valor: almacena el número que queremos insertar, eliminar o buscar.